p8106_hw3_sc5826

Shivalika Chavan

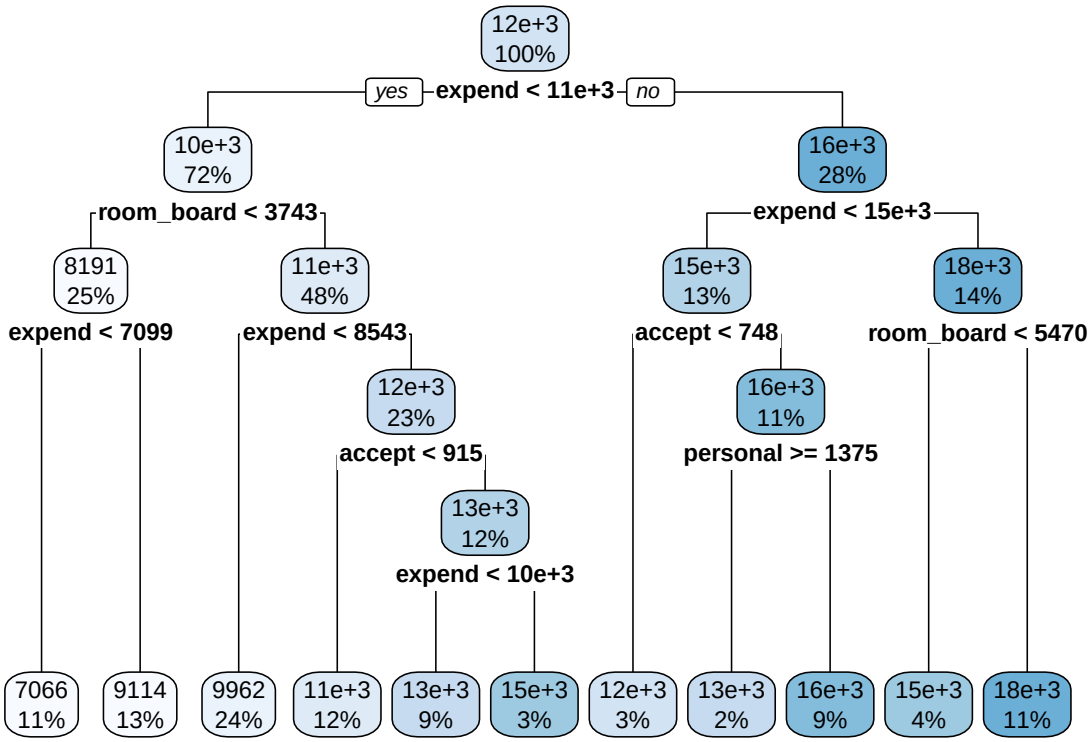
2026-03-28

1

```
college = read_csv("./data/College.csv") |>
  janitor::clean_names() |>
  select(-college)

set.seed(1234)
data_split = initial_split(college, prop = 0.8)

training_data = training(data_split)
testing_data = testing(data_split)
n_param = ncol(college) - 1
```

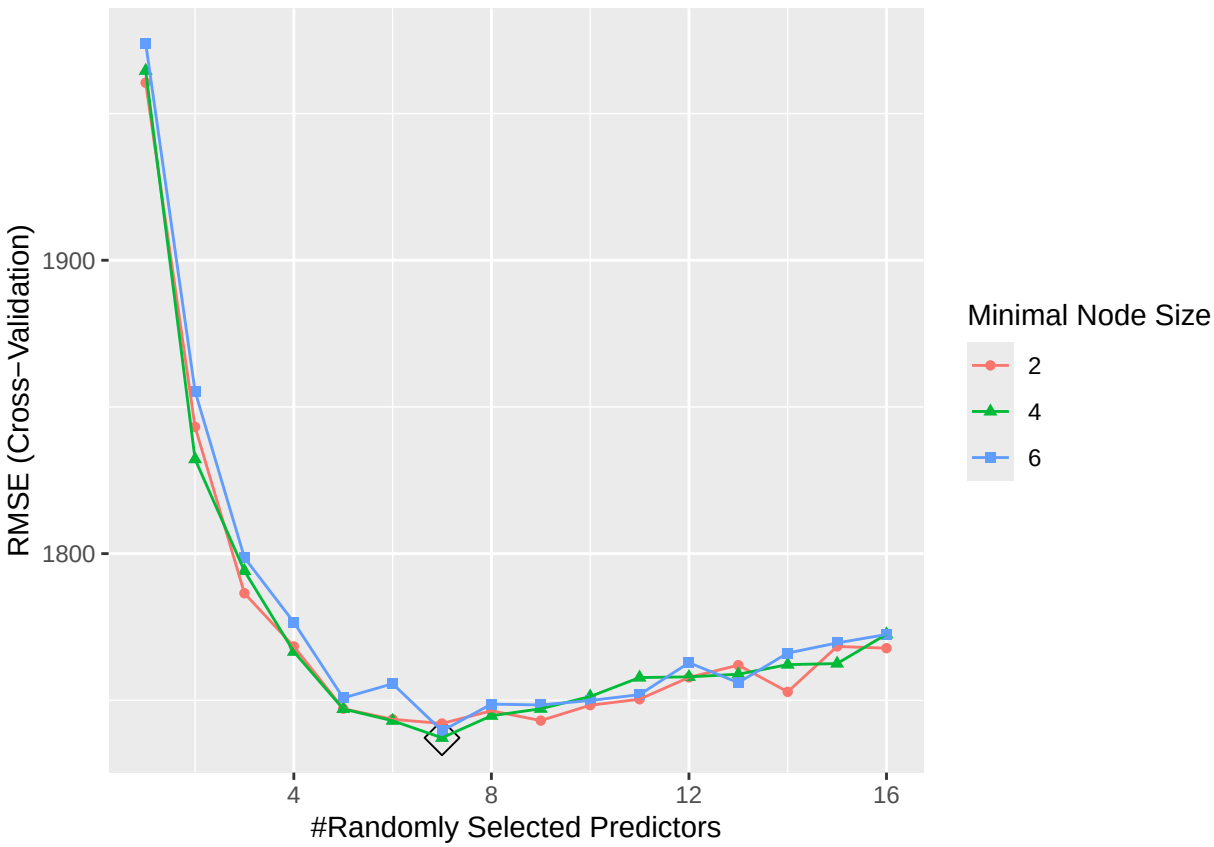
1a - Regression Tree

```
tree1 = rpart(formula = outstate ~ . ,
              data = training_data,
              control = rpart.control(cp = 0.008))

rpart.plot(tree1)
```

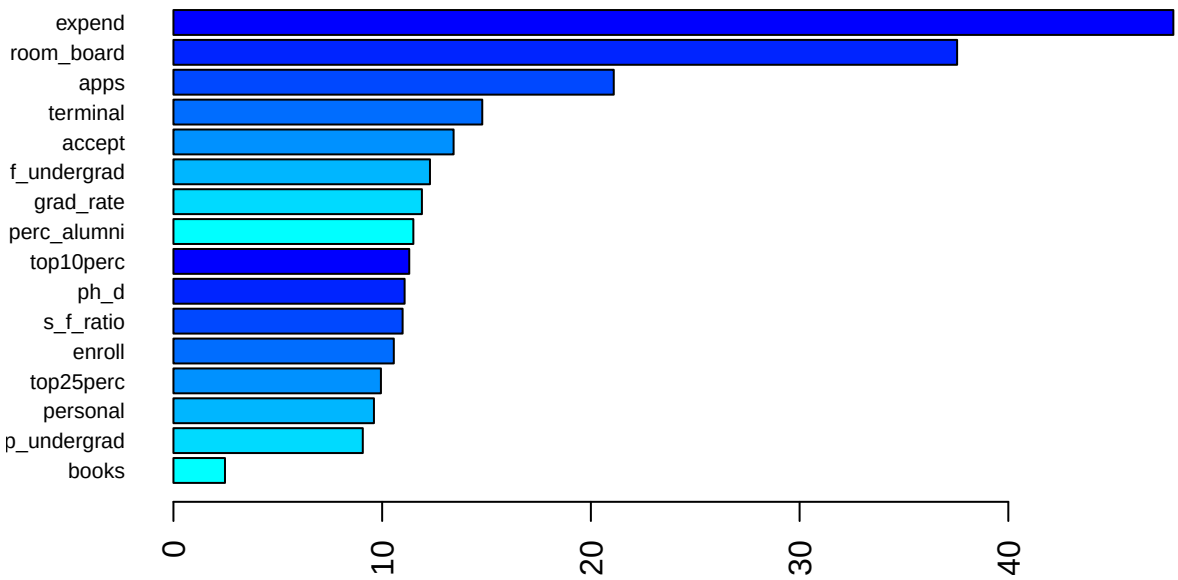


1b - Random Forest

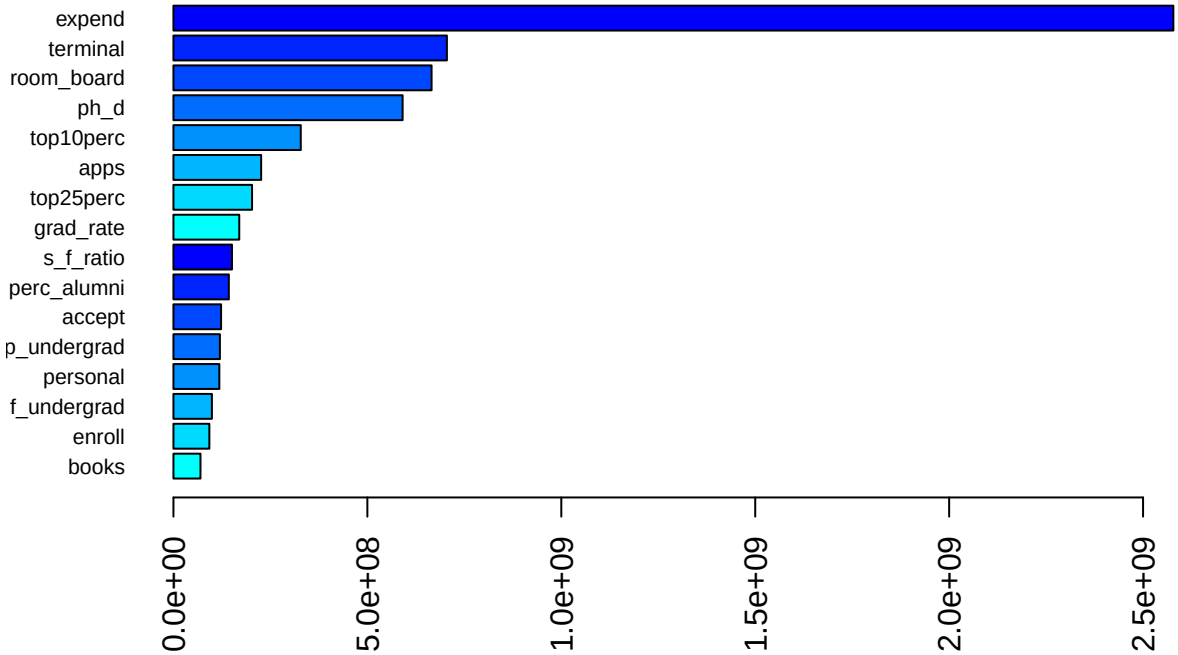
```
ctrl = trainControl(method = "cv", number = 10)

rf.grid = expand.grid(mtry = 1:n_param, # test all possible which is 1 to p (p = 16)
                      splitrule = "variance", # split based on rss, no need to change
                      min.node.size = c(2, 4, 6)) # smallest # of obs in each node
set.seed(1234)
rf.fit = train(outstate ~ . ,
               data = training_data,
               method = "ranger", # better to use fast implementation
               tuneGrid = rf.grid,
               trControl = ctrl)

ggplot(rf.fit, highlight = TRUE)
```




```
set.seed(1234)
rf.final.per = ranger(outstate ~ . ,
                      training_data,
                      mtry = rf.fit$bestTune[[1]],
                      min.node.size = rf.fit$bestTune[[3]],
                      splitrule = "variance",
                      importance = "permutation",
                      scale.permutation.importance = TRUE)

barplot(sort(ranger::importance(rf.final.per), decreasing = FALSE),
        las = 2, horiz = TRUE, cex.names = 0.7,
        col = colorRampPalette(colors = c("cyan","blue"))(8))
```




```
set.seed(1234)
rf.final.imp = ranger(outstate ~ . ,
                      training_data,
                      mtry = rf.fit$bestTune[[1]],
                      splitrule = "variance",
                      min.node.size = rf.fit$bestTune[[3]],
                      importance = "impurity")

barplot(sort(ranger::importance(rf.final.imp), decreasing = FALSE),
        las = 2, horiz = TRUE, cex.names = 0.7,
        col = colorRampPalette(colors = c("cyan","blue"))(8))
```

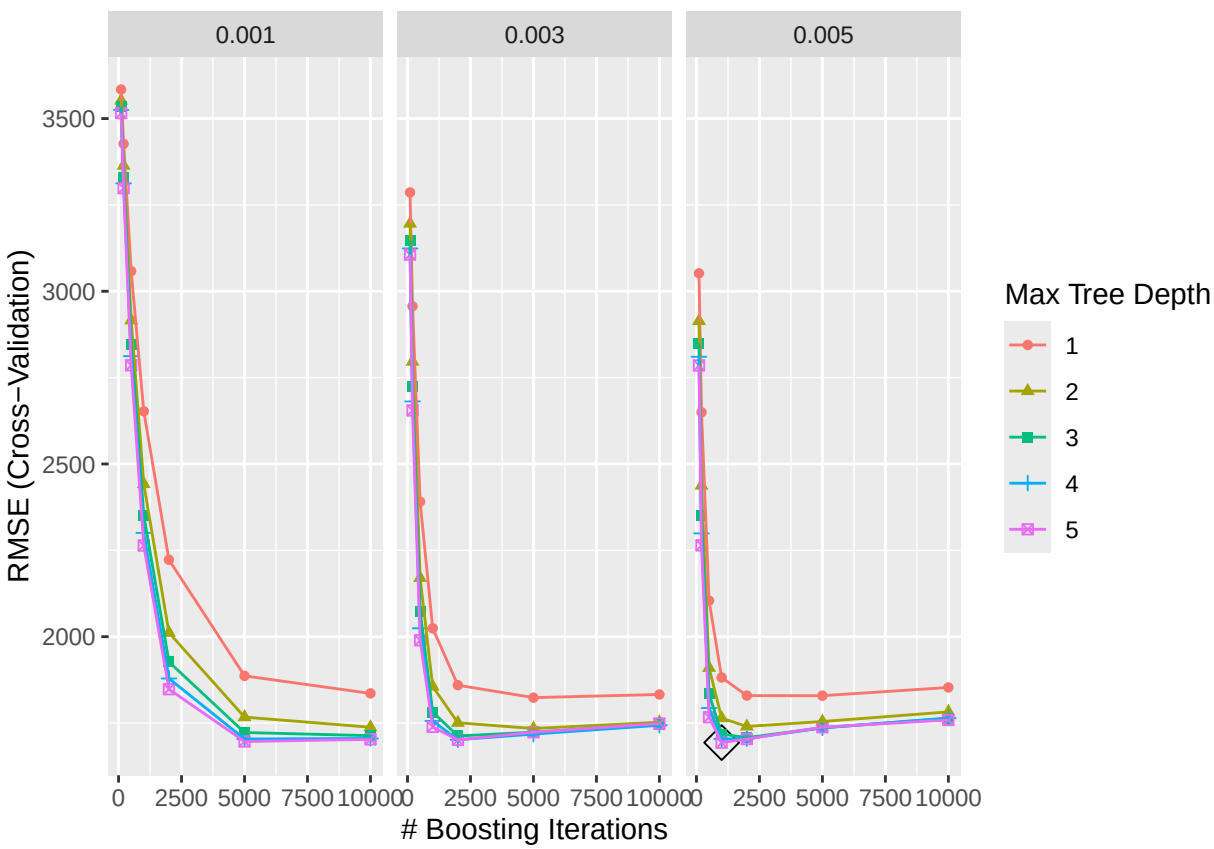



```
rf_predictions = predict(rf.fit, newdata = testing_data)
test_rmse = RMSE(pred = rf_predictions, obs = testing_data$outstate)
```

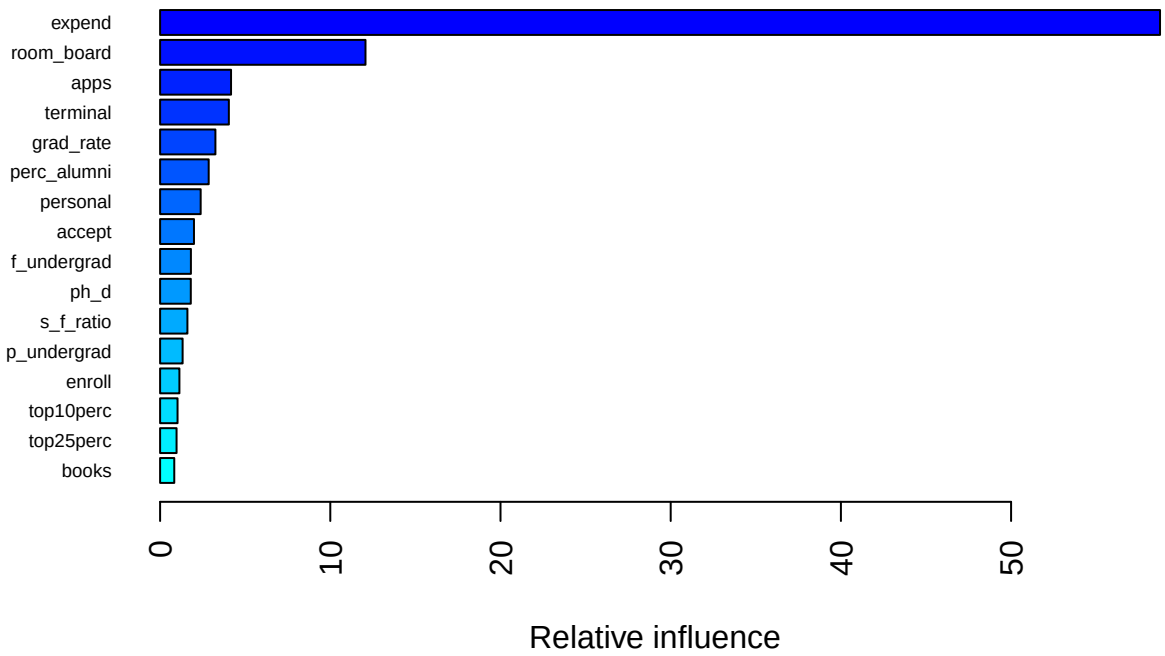
The tuned RF model had a test RMSE of 1702.1. Looking at the top 5 variables from both permutation and impurity importance metrics, instructional expenditure per student (`expend`), room and board costs (`room_board`), and percentage of faculty with terminal degrees (`terminal`) all affect tuition costs.

1c - Boosting

```
gbmA.grid = expand.grid(n.trees = c(100,200,500,1000,2000,5000,10000),
                        interaction.depth = 1:5,
                        shrinkage = c(0.001, 0.003, 0.005),
                        n.minobsinnode = 10)
set.seed(1234)
gbmA.fit = train(outstate ~ . ,
                 training_data,
                 tuneGrid = gbmA.grid,
                 trControl = ctrl,
                 method = "gbm",
                 distribution = "gaussian",
                 metric = "RMSE",
                 verbose = FALSE)

ggplot(gbmA.fit, highlight = TRUE)
```




```
summary(gbmA.fit$finalModel, las = 2, cBars = 19, cex.names = 0.6)
```




```
##                     var    rel.inf
## expend           expend 58.7502789
## room_board   room_board 12.0669857
## apps               apps  4.1673202
## terminal       terminal  4.0384661
## grad_rate     grad_rate  3.2501969
## perc_alumni perc_alumni  2.8552941
## personal       personal  2.3826842
## accept           accept  1.9893766
## f_undergrad f_undergrad  1.8102317
## ph_d               ph_d  1.8041146
## s_f_ratio     s_f_ratio  1.6049518
## p_undergrad p_undergrad  1.3222257
## enroll           enroll  1.1345520
## top10perc     top10perc  1.0243951
## top25perc     top25perc  0.9664886
## books             books  0.8324378
```

```
gbmA_predictions = predict(gbmA.fit, newdata = testing_data)
test_rmse = RMSE(pred = gbmA_predictions, obs = testing_data$outstate)
```

The tuned model had a test RMSE of 1676.08. Looking at the top 5 variables from this model, instructional expenditure per student (`expend`), room and board costs (`room_board`), and percentage of faculty with terminal degrees (`terminal`) all affect tuition costs. This is in agreement with the random forest models in the previous section.

2

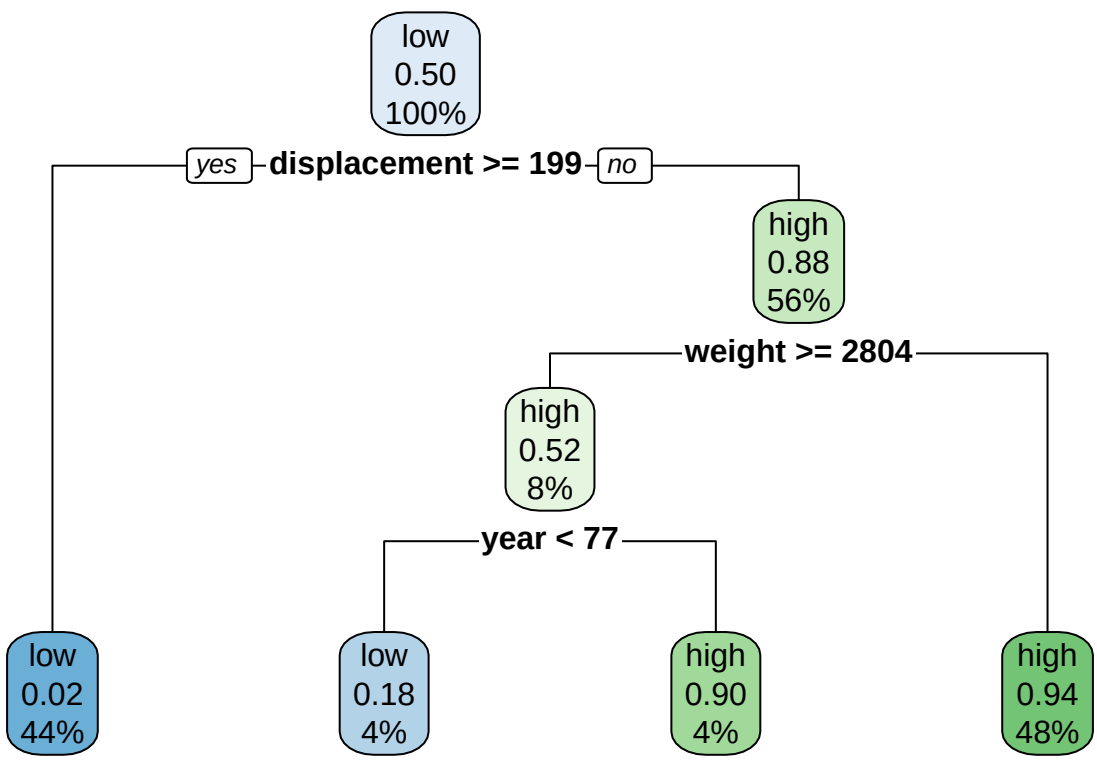
Data Cleaning and Test/Train Split:

```
auto_data = read_csv("./data/auto.csv") |>
  mutate(
    origin = factor(origin, levels = c("1", "2", "3"),
                    labels = c("American", "European", "Japanese")),
```

```r
    mpg_cat = factor(mpg_cat, levels = c("low", "high"))
  )

set.seed(1234)
indexTrain = createDataPartition(y = auto_data$mpg_cat, p = 0.7, list = FALSE)
auto_train = auto_data[indexTrain, ]
auto_test = auto_data[-indexTrain, ]

n_param = ncol(auto_data) - 1
```

2a - Classification Tree

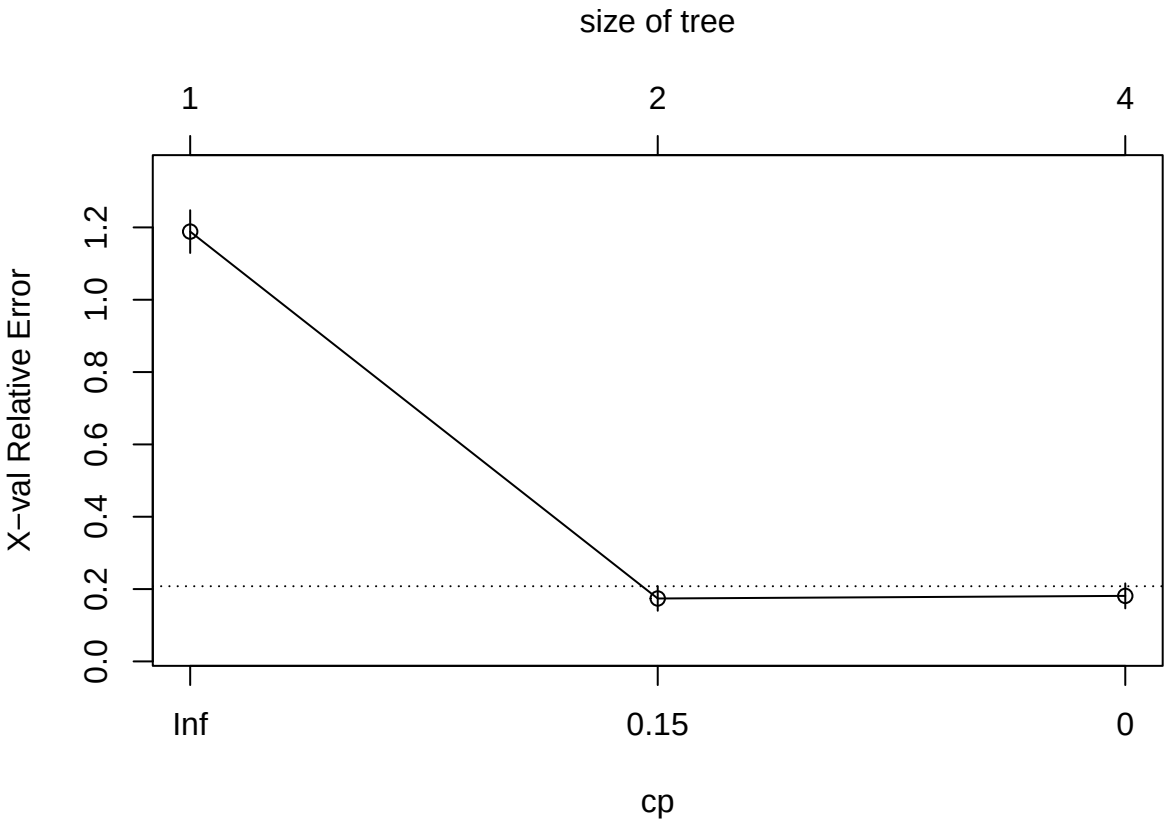
```r
set.seed(1234)
tree1 = rpart(formula = mpg_cat ~ . ,
              data = auto_train,
              control = rpart.control(cp = 0))
rpart.plot(tree1) # biggest possible tree
```



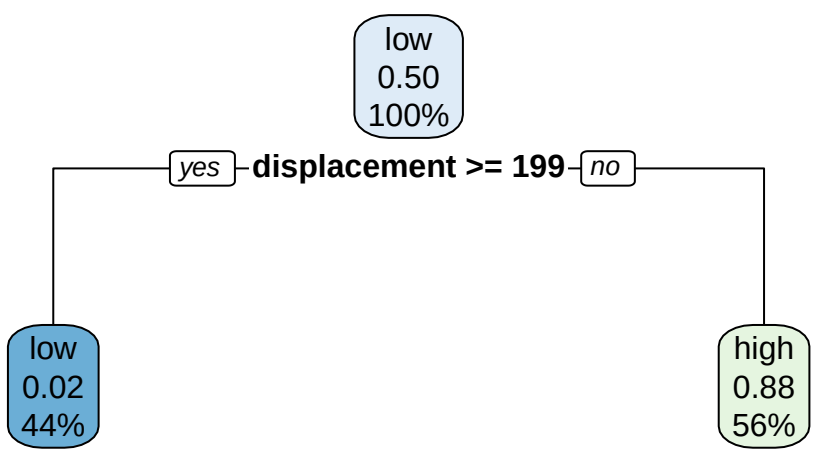

```r
cptable = tree1$cptable |> as.data.frame()
cptable |> knitr::kable(digits = 4)
```

CP	nsplit	rel error	xerror	xstd
0.8551	0	1.0000	1.1884	0.0591
0.0254	1	0.1449	0.1739	0.0339
0.0000	3	0.0942	0.1812	0.0346

```r
plotcp(tree1)
```




```r
# minimum cross-validation error; may also use the 1SE rule
minErr = which.min(cptable$xerror)
tree_min = rpart::prune(tree1, cp = cptable$CP[minErr])
rpart.plot(tree_min)
```



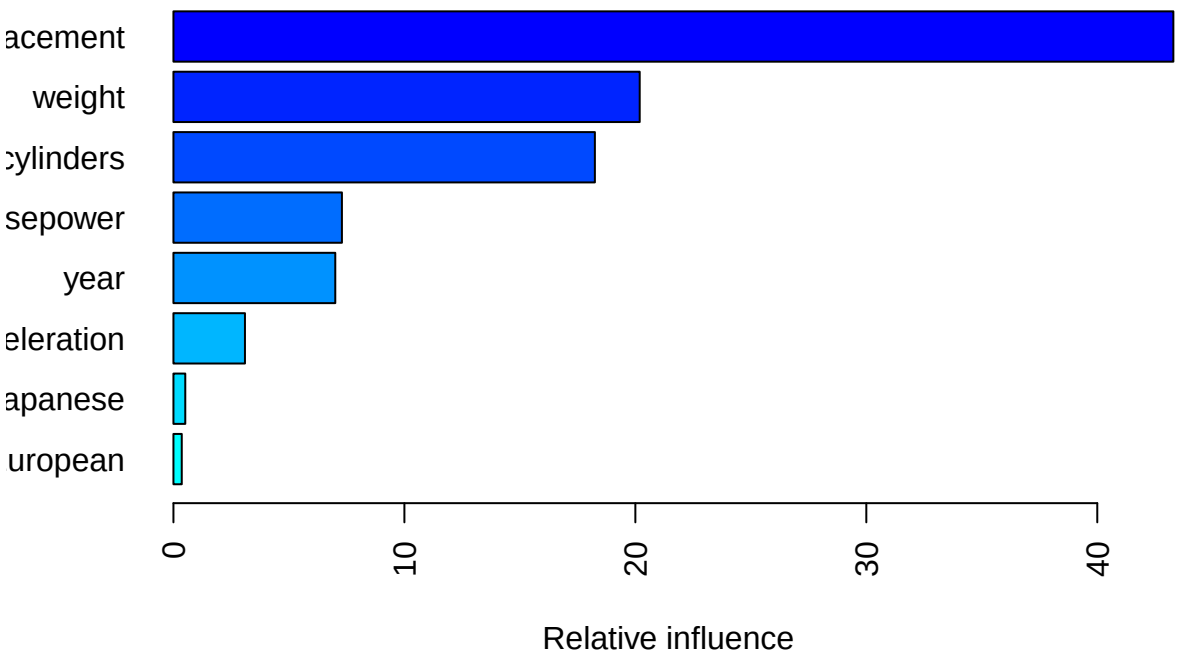

```r
# 1SE Rule
threshold = cptable$xerror[minErr] + cptable$xstd[minErr]
se_row = min(which(cptable$xerror <= threshold))
tree_1se = rpart::prune(tree1, cp = cptable$CP[se_row])
rpart.plot(tree_1se)
```

The tree size corresponding to both lowest cross-validation error and the 1SE rule is 2. This makes sense

since there are so few predictors to begin with.

2b - Boosting

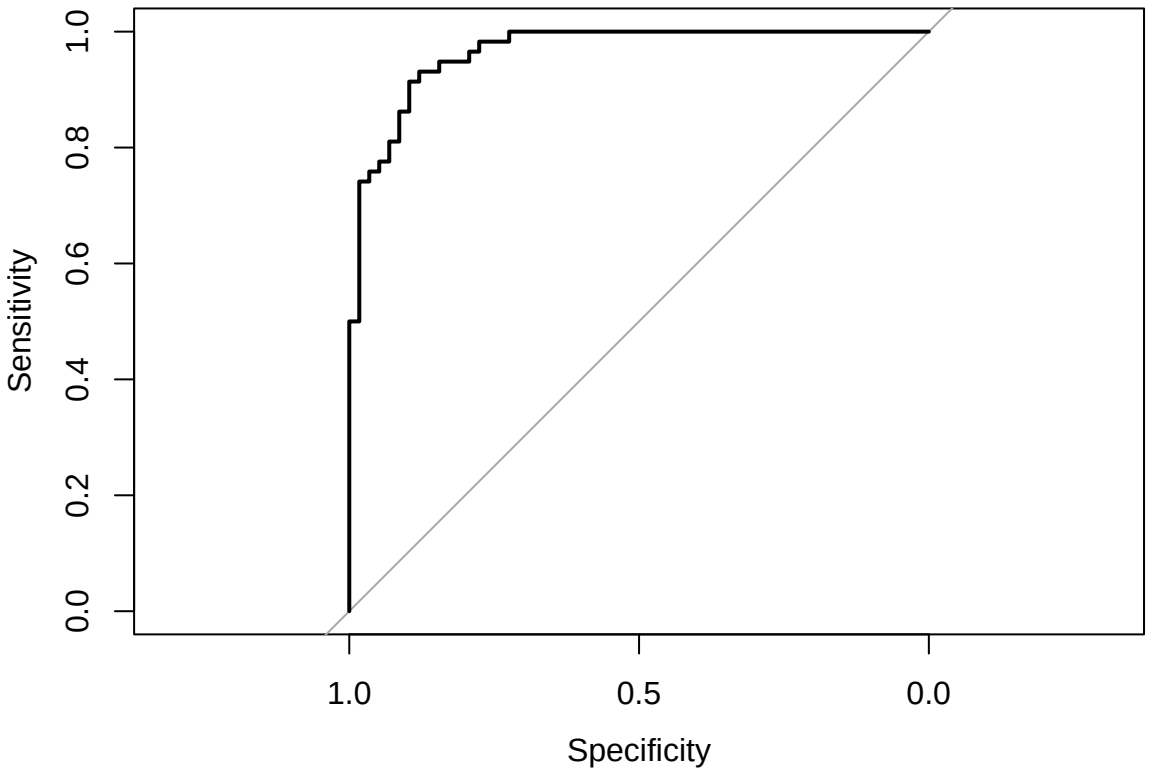
```
gbm.grid = expand.grid(n.trees = c(100, 500, 1000, 2000),
                       interaction.depth = 1:5,
                       shrinkage = c(0.001, 0.005, 0.01),
                       n.minobsinnode = 10)

ctrl = trainControl(method = "cv",
                    classProbs = TRUE,
                    summaryFunction = twoClassSummary)

set.seed(1234)
model_gbm = train(mpg_cat ~ .,
                  data = auto_train,
                  method = "gbm",
                  distribution = "bernoulli",
                  metric = "ROC",
                  tuneGrid = gbm.grid,
                  trControl = ctrl,
                  verbose = FALSE)

summary(model_gbm$finalModel, las = 2, cBars = 10)
```




```
##                          var    rel.inf
## displacement     displacement 43.2934766
## weight                 weight 20.1878277
## cylinders           cylinders 18.2471289
## horsepower         horsepower  7.2945631
## year                     year  7.0074238
## acceleration     acceleration  3.0963033
## originJapanese originJapanese  0.5125313
```

```
## originEuropean originEuropean  0.3607453
```

```r
gbm_probs = predict(model_gbm, newdata = auto_test, type = "prob")$high
gbm_roc = roc(auto_test$mpg_cat, gbm_probs)
plot(gbm_roc)
```




```r
auc(gbm_roc)
```

```
## Area under the curve: 0.9652
```

The model achieved an AUC of 0.97 on the test dataset. This means the model can distinguish well between cars with high and low predicted gas mileage.